

What is an algorithm?

A set of steps to accomplish the task

It is step by step action to solve the problem.

An [algorithm](#) is a procedure used for solving a problem or performing a computation. Algorithms act as an exact list of instructions that conduct specified actions step by step in either hardware- or software-based [routines](#).

Algorithms are widely used throughout all areas of IT. In mathematics, computer programming and computer science, an algorithm usually refers to a small procedure that solves a recurrent problem.

Algorithms are also used as specifications for performing data processing and play a major role in automated systems.

An algorithm is a self-contained step-by-step set of operations to be performed to solve a specific problem or a class of problems.

For example, an **algorithm** for driving to a friend's house could be:

1. Find your keys
2. Walk out of the house
3. Close the door
4. Open the car door
5. Get into the car
6. Put the key into the ignition, etc.....

An [algorithm](#) is a precise step-by-step plan for a computational procedure that possibly begins with an input value and yields an output value in a finite number of steps in order to [solve a particular problem](#)

An algorithm could be used for sorting sets of numbers or for more complicated tasks, such as recommending user content on [social media](#). Algorithms typically start with initial input and instructions that describe a specific computation. When the computation is executed, the process produces an output.

How do algorithms work?

Algorithms work by following a set of instructions or rules to complete a task or solve a problem. They can be expressed as [natural languages](#), programming languages, [pseudocode](#), flowcharts and control tables. Natural language expressions are rare, as they are more ambiguous. Programming languages are normally used for expressing algorithms executed by a computer.

Algorithms use an initial input along with a set of instructions. The input is the initial data needed to make decisions and can be represented in the form of numbers or words. The input data gets put through a set of instructions, or computations, which can include arithmetic and decision-making processes. The output is the last step in an algorithm and is normally expressed as more data.

For example, a search algorithm takes a search query as input and runs it through a set of instructions for searching through a [database](#) for relevant items to the query. Automation software acts as another example of algorithms, as automation follows a set of rules to complete tasks. Many algorithms make up automation software, and they all work to automate a given process.

Algorithm to find whether a given number is odd or even : Even integers can always be divided into two without leaving any remaining pieces. They have a 0, 2, 4, 6, or 8 digit ending. Odd numbers are not equally divided by 2. After division by an integer, the

remaining operation, percent, returns the residual. If n is even, $n \text{ percent } 2 == 0$ will indeed be true, but if n is odd, it will also be incorrect.

ALGORITHM :

STEP 1: START

STEP 2: Receive data from a user or dynamically assign value.

STEP 3: if $\text{num percent } 2 == 0$. Even Integer should be printed.

STEP 4: Else, Print an Odd Number

STEP 5: Print an Odd Number

STEP 6: END

Step 1: Start

Step 2: Read a number to N

Step 3: Divide the number by 2 and store the remainder in R .

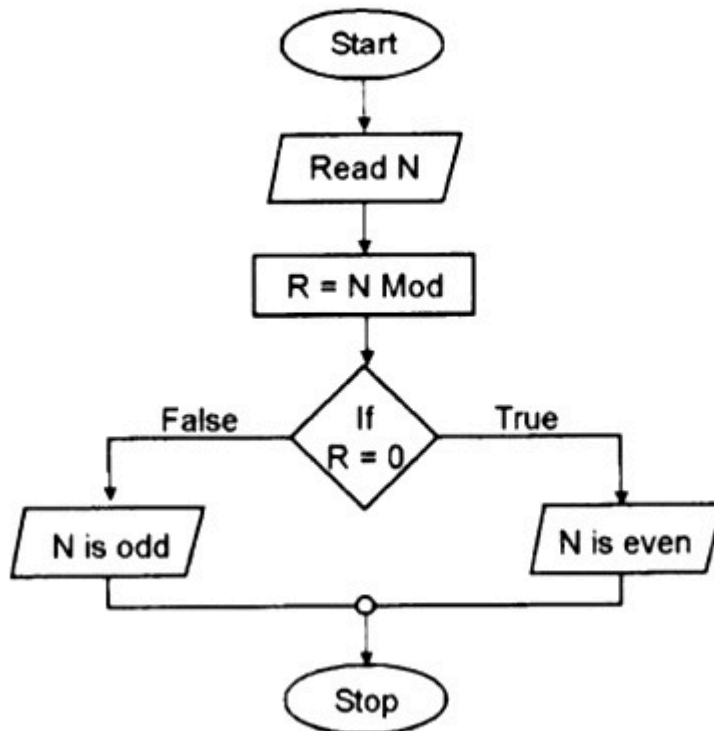
Step 4: If $R = 0$ Then go to Step 6

Step 5: Print "N is odd" go to step 7

Step 6: Print "N is even"

Step 7: stop

Flow chart to check whether the given number is even or odd.



Flowcharts are nothing but the graphical representation of the data or the algorithm for a better understanding of the code visually.

It displays step-by-step solutions to a problem, algorithm, or process.

It is a pictorial way of representing steps that are preferred by most beginner-level programmers to understand algorithms of computer science, thus it contributes to troubleshooting the issues in the algorithm.

A flowchart is a picture of boxes that indicates the process flow sequentially. Since a flowchart is a pictorial representation of a process or algorithm, it's easy to interpret and understand the process.

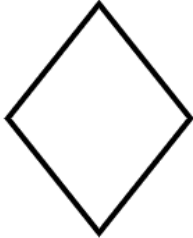
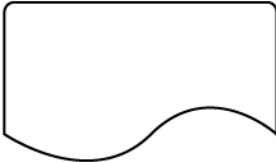
To draw a flowchart, certain rules need to be followed which are followed by all professionals to draw a flowchart and are widely accepted all over the countries.

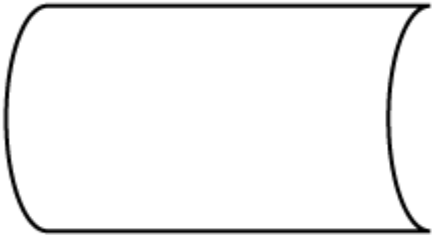
What is FlowChart?

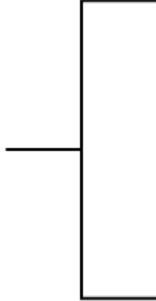
A flowchart is a type of diagram that represents a workflow or process. A flowchart can also be defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task.

Flowchart symbols

Different types of boxes are used to make flowcharts flowchart Symbols. All the different kinds of boxes are connected by arrow lines. Arrow lines are used to display the flow of control. Let's learn about each box in detail.

Symbol Name	Symbol Representation
Terminal/Terminator	
Process	
Decision	
Document	

Symbol Name	Symbol Representation
Data or Input/Output	
Stored Data	
Flow Arrow	

Symbol Name	Symbol Representation
Comment or Annotation	
Predefined process	
On-page connector/reference	

Uses of Flowcharts in Computer Programming/Algorithms

The following are the uses of a flowchart:

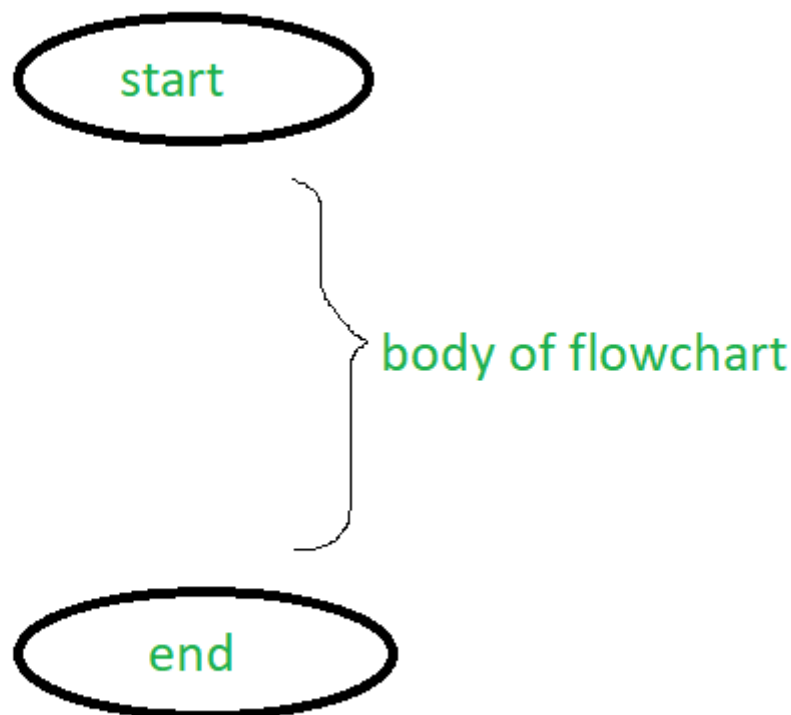
- It is a pictorial representation of an algorithm that increases the readability of the program.
- Complex programs can be drawn in a simple way using a flowchart.
- It helps team members get an insight into the process and use this knowledge to collect data, detect problems, develop software, etc.

- A flowchart is a basic step for designing a new process or adding extra features.
- Communication with other people becomes easy by drawing flowcharts and sharing them.

When to Use Flowchart?

Flowcharts are mainly used in the below scenarios:

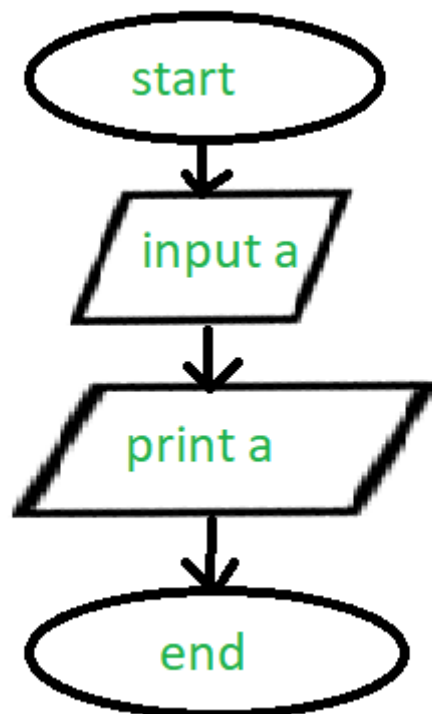
- It is most importantly used when programmers make projects. As a flowchart is a basic step to make the design of projects pictorially, it is preferred by many.
- When the flowcharts of a process are drawn, the programmer understands the non-useful parts of the process. So flowcharts are used to separate sound logic from the unwanted parts.
- Since the rules and procedures of drawing a flowchart are universal, a flowchart serves as a communication channel to the people who are working on the same project for better understanding.
- Optimizing a process becomes easier with flowcharts. The efficiency of the code is improved with the flowchart drawing.



2. Data



This is a parallelogram-shaped box inside which the inputs or outputs are written. This basically depicts the information that is entering the system or algorithm and the information that is leaving the system or algorithm. For example: if the user wants to input *a* from the user and display it, the flowchart for this would be:

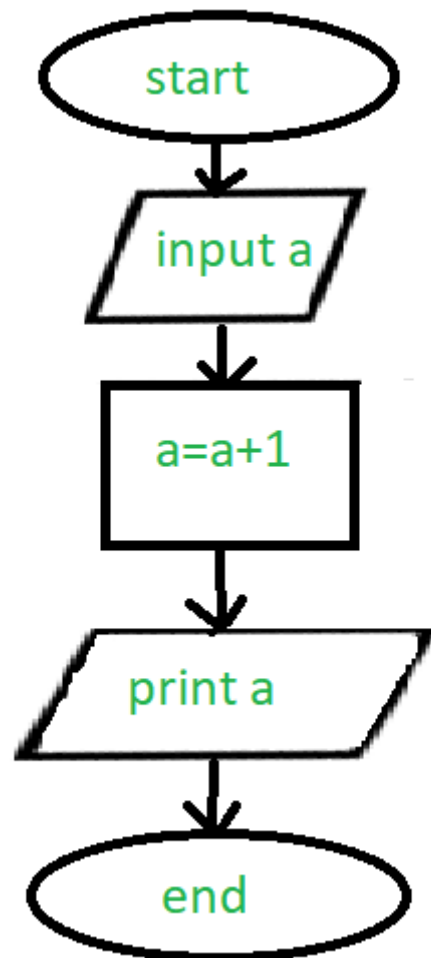


3. Process

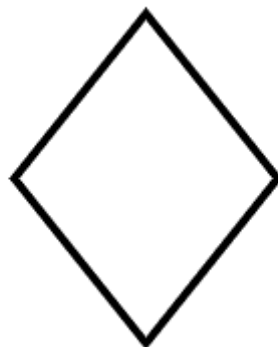


This is a rectangular box inside which a programmer writes the main course of action of the algorithm or the main logic of the program. This is the crux of the flowchart as the main processing codes is written inside this box. For

example: if the programmer wants to add 1 to the input given by the user, he/she would make the following flowchart:

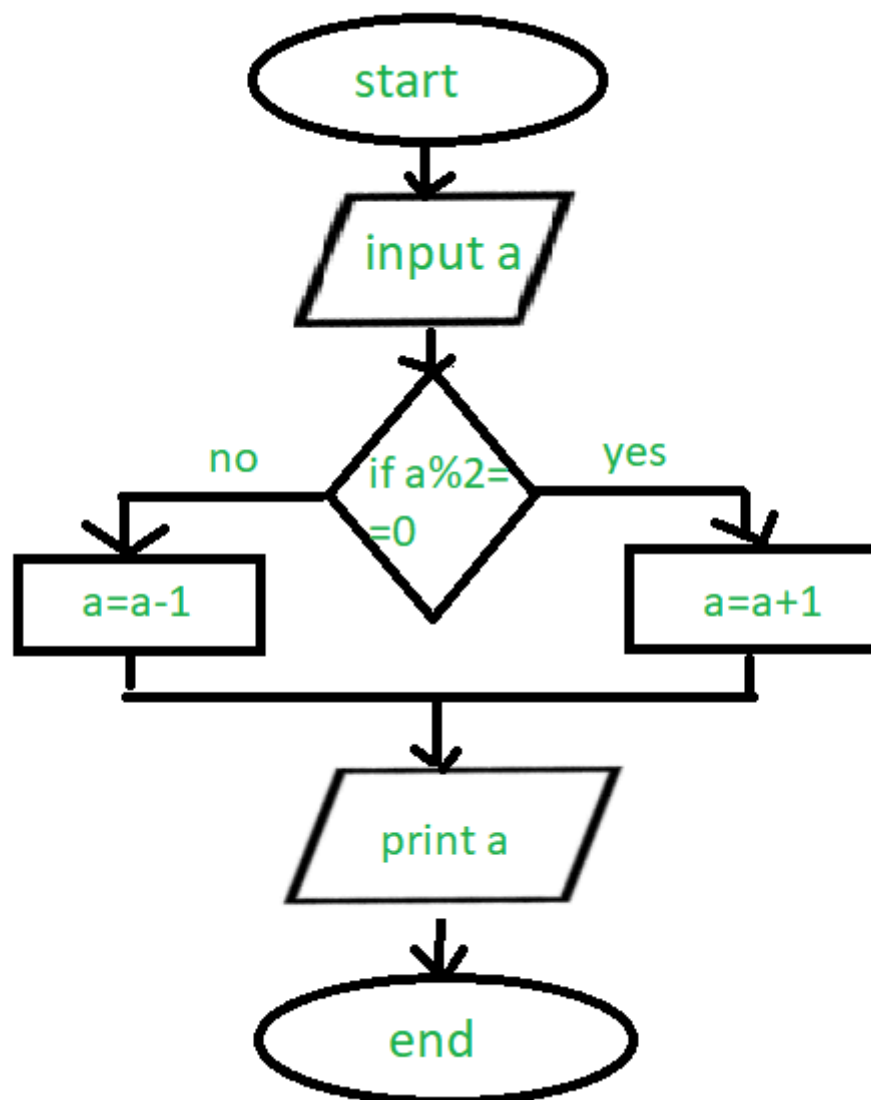


4. Decision



This is a rhombus-shaped box, control statements like if, condition like $a > 0$, etc are written inside this box. There are 2 paths from this one which is "yes" and the other one is "no". Every decision has either yes or no as an option,

similarly, this box has these as options. For example: if the user wants to add 1 to an even number and subtract 1 if the number is odd, the flowchart would be:



5. Flow

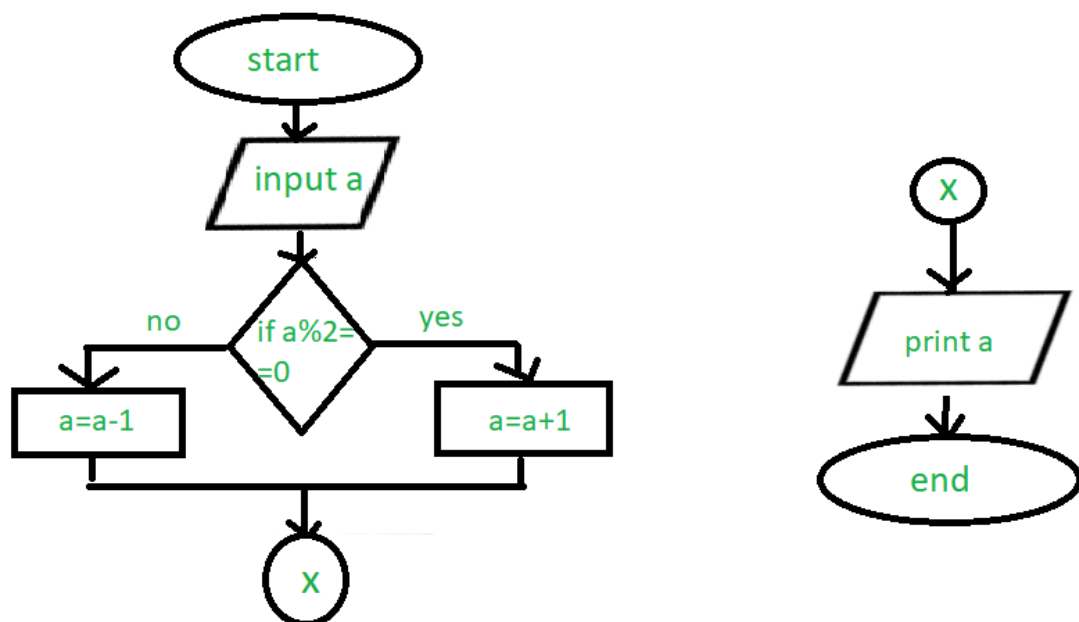


This arrow line represents the flow of the algorithm or process. It represents the direction of the process flow. In all the previous examples, we included arrows in every step to display the flow of the program. An arrow increases the readability of the program.

6. On-Page Reference



This circular figure is used to depict that the flowchart is in continuation with the further steps. This figure comes into use when the space is less and the flowchart is long. Any numerical symbol is present inside this circle and that same numerical symbol will be depicted before the continuation to make the user understand the continuation. Below is a simple example depicting the use of On-Page Reference



Advantages of Flowchart

- It is the most efficient way of communicating the logic of the system.
- It acts as a guide for a blueprint during the program design.
- It also helps in the debugging process.
- Using flowcharts we can easily analyze the programs.
- flowcharts are good for documentation.

Disadvantages of Flowchart

- Flowcharts are challenging to draw for large and complex programs.
- It does not contain the proper amount of details.
- Flowcharts are very difficult to reproduce.
- Flowcharts are very difficult to modify.

Solved Examples on FlowChart

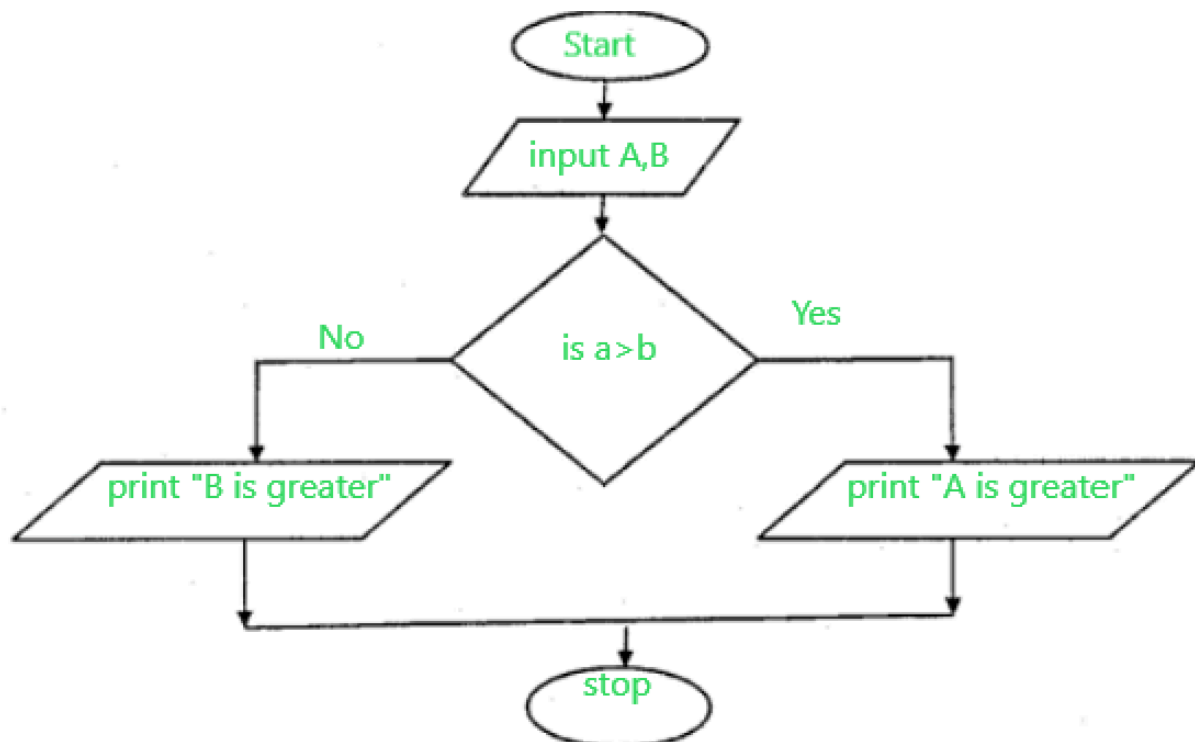
Question 1. Draw a flowchart to find the greatest number among the 2 numbers.

Solution:

Algorithm:

1. Start
2. Input 2 variables from user
3. Now check the condition If $a > b$, goto step 4, else goto step 5.
4. Print a is greater, goto step 6
5. Print b is greater
6. Stop

FlowChart:



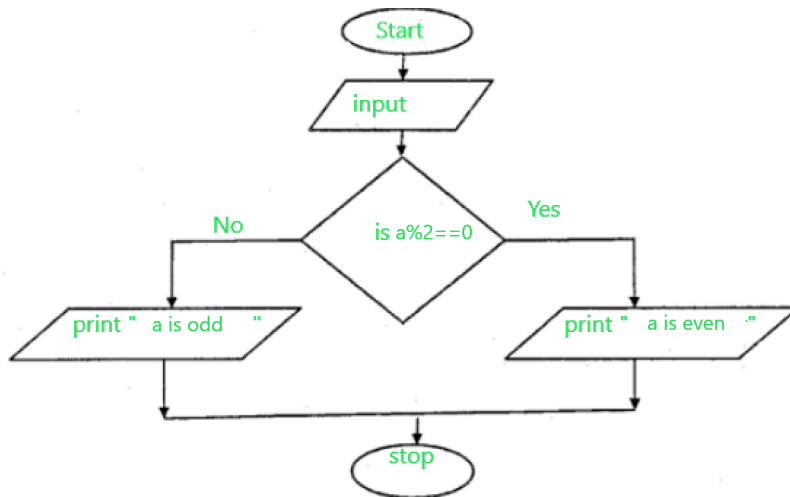
Question 2. Draw a flowchart to check whether the input number is odd or even

Solution:

Algorithm:

1. Start
2. Put input a
3. Now check the condition if $a \% 2 == 0$, goto step 5. Else goto step 4
4. Now print("number is odd") and goto step 6
5. Print("number is even")
6. Stop

FlowChart:



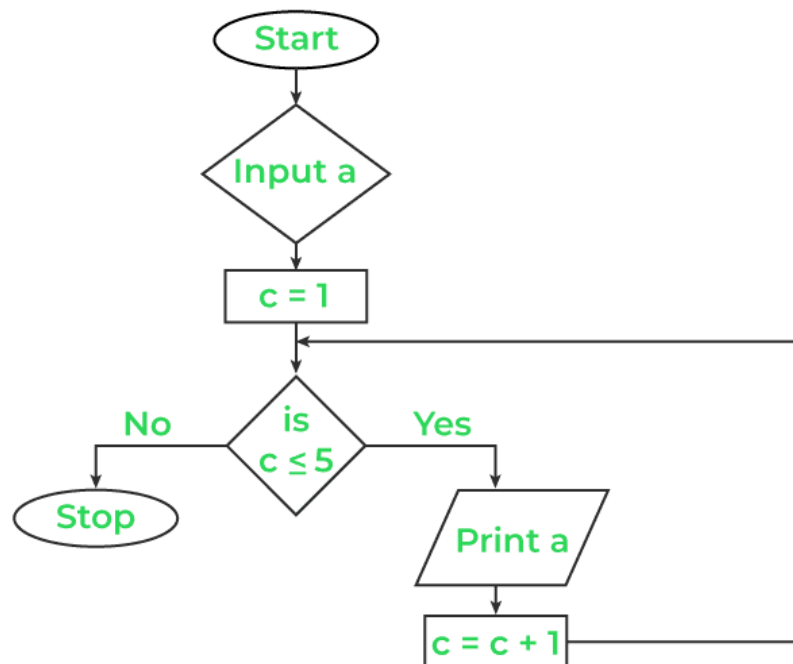
Question 3. Draw a flowchart to print the input number 5 times.

Solution:

Algorithm:

1. Start
2. Input number a
3. Now initialise $c = 1$
4. Now we check the condition if $c \leq 5$, goto step 5 else, goto step 7.
5. Print a
6. $c = c + 1$ and goto step 4
7. Stop

FlowChart:



Flowchart to print the input number 5 times